

CineScope: 26-Week Computer Vision Roadmap

2026-09-20 · @Someone

Overview

In 26 weeks (about 10–15 hours a week) you build CineScope: a system that watches a Hollywood film and outputs a character timeline, a shot-by-shot breakdown, color palettes, plain-English scene search, VFX tools and a 3D fly-through of a set. Every phase ships a visible feature, so you have a demo after each one.

The rule for every phase: implement the core idea from scratch first, then switch to the production library and compare. You learn the idea from your own code, and the library comparison gives you real numbers to talk about.

Phase	Weeks	Feature it adds	CV area
0	1–3	Shot cuts, camera moves, color palettes	Classical CV
1	4–5	Shot-framing labels	CNNs
2	6–8	Face and prop detection	Detection
3	9–11	Character timeline, who's talking	Tracking, metric learning
4	12–13	Actor cutouts, object removal	Segmentation, matting
5	14–17	Camera path, 2D-to-3D, set fly-through	3D vision
6	18–20	Fight analysis, scene types	Pose, video transformers
7	21–22	Subtitles and credits read	OCR
8	23–24	Scene search, chat with the movie	Multimodal retrieval
9	25–26	Fast pipeline, web app, launch	Deployment

Stack: Python, NumPy, PyTorch, OpenCV (only after you've written the algorithm yourself), PyAV or decord for video, FAISS or Qdrant for vectors, FastAPI plus a simple React front end.

Compute: Kaggle gives about 30 free GPU hours a week, and Colab covers the rest. Phases 5 and 8 are the heaviest. Work on 1–3 minute clips and only run full films once the pipeline is fast.

Repo layout: one repo, one folder per phase (`p0_classical/` , `p1_cnn/` ...), each with its own README showing a GIF, a metric and what you learned. A shared `cinescope/` package holds the pipeline code the app uses.

Footage: develop on trailers and short clips. For anything public (README GIFs, demo video), use open films like Blender's *Tears of Steel* (live action with real VFX) and *Sintel*, plus short excerpts from Hollywood films.

Core references for the whole project: Stanford CS231n (lectures and assignments), Szeliski's *Computer Vision: Algorithms and Applications* (2nd ed., free online), and Hartley & Zisserman's *Multiple View Geometry* for Phase 5.

Phase 0 — Classical CV (Weeks 1–3)

By week 3, CineScope splits a trailer into shots, labels each shot's camera move and shows the film's color palette, all with NumPy and no deep learning.

Week	Build	Read
1	Video I/O with PyAV; convolution, Gaussian blur and Sobel filters in NumPy; color spaces (RGB, HSV, Lab)	Szeliski ch. 3; CS231n image basics
2	Shot-cut detection from frame-to-frame color-histogram distance, plus a fade/dissolve detector; k-means color palettes per shot, rendered as a "movie barcode" strip	TransNet V2 paper (for the baseline you'll compare against later)
3	Harris corners, Lucas-Kanade optical flow and RANSAC homography, all from scratch; classify each shot as static, pan, tilt, zoom or handheld	Szeliski ch. 7–9; Lucas & Kanade (1981)

Data: 5 trailers of your choice. Hand-label the cuts in 2 of them (about 150 cuts) as your test set.

Done when: cut detection reaches $F1 \geq 0.85$ on your labeled trailers, and the palette strip for a film like *Mad Max: Fury Road* looks right at a glance.

Interview questions to master: Why does Lab beat RGB for color distance? What does RANSAC assume, and how many iterations do you need? Why do gradual transitions (dissolves) break histogram-based cut detection?

Phase 1 — CNN fundamentals (Weeks 4–5)

By week 5, every shot gets a framing label (close-up, medium, wide and so on) from a ResNet you wrote yourself.

Week	Build	Read
4	A tiny autograd engine (micrograd-style), then a conv layer with backprop in NumPy; check your gradients numerically against PyTorch	Karpathy's micrograd video; CS231n assignments 1–2
5	ResNet-18 from scratch in PyTorch; train a shot-scale classifier on MovieShots; add augmentation, a cosine learning-rate schedule and mixed precision; compare ImageNet-pretrained vs. from scratch	He et al., ResNet (2015); Ioffe & Szegedy, BatchNorm (2015); Rao et al., MovieShots (ECCV 2020)

Data: MovieShots (about 46k shots from movie trailers with framing and movement labels).

Done when: your pretrained ResNet gets within a few points of the accuracy reported in the MovieShots paper, and you can show a confusion matrix explaining which framings get mixed up.

Interview questions to master: Why do residual connections make deep networks trainable? What does BatchNorm do at train vs. inference time? Your loss is flat at epoch 1 — what do you check first?

Phase 2 — Detection (Weeks 6–8)

By week 8, CineScope finds every face and key prop in a clip, using a detector you built and evaluated with your own metric code.

Week	Build	Read
6	IoU, NMS and COCO-style mAP from scratch; unit-test them against pycocotools	Szeliski ch. 6; COCO evaluation docs
7	A CenterNet-style anchor-free face detector (heatmap + box size + offset heads) on a ResNet backbone with an FPN; train on WIDER FACE	Zhou et al., <i>Objects as Points</i> (2019); Lin et al., FPN (2017)
8	Compare against a production detector (YOLO or RT-DETR) on COCO classes like cars, guns and phones; read DETR and explain the difference; run both on movie clips	Carion et al., DETR (2020); Tian et al., FCOS (2019)

Data: WIDER FACE for faces; COCO for props.

Done when: your face detector hits a respectable AP on the WIDER FACE Easy split, and you have a table comparing speed and accuracy against the library model on 3 movie clips (dark scenes, profile shots, crowds).

Interview questions to master: Anchor-based vs. anchor-free — trade-offs? Why does NMS fail in crowds, and what replaces it? Walk through how $mAP@0.5:0.95$ is calculated.